

TKS / Tootra — Master README + CLI Runbook

Generated: 2026-01-04 09:00 UTC

This document is the **single operational manual** for building and running the TKS/Tootra agent system via CLI agents.

It assumes you are implementing the 7 architecture modules already spec'd (Executor/Router, Verifier, Capability Registry, Outcome Scoring, Eval/Replay, Observability/Audit Log, Governance Rails).

0) What you are building (one sentence)

A goal-driven agent that turns **RPM prerequisites** into **tool actions**, while preserving meaning via **TKS expressions**, staying safe via **High-Stakes gates**, and continuously improving via **reward + replayable evals**.

1) Minimum viable system (MVS) — the first thing that must run

The system is “alive” when this loop works end-to-end in a sandbox:

1. **Input**: goal + context + Foundation stack
2. **RPM**: produce prerequisite tree (depth ≤ 2 initially)
3. **Router**: choose ActionType/Tool for each prerequisite
4. **Executor**: run tools (stub tools allowed at first)
5. **Verifier**: parse + evidence bind + consistency + gate (ALLOW/PAUSE/BLOCK)
6. **Memory write**: evidence node + TKS packet
7. **Outcome scoring**: reward + penalties
8. **Logs**: event stream from input→output
9. **Replay**: re-run same episode and compare results

If any step is missing, your system cannot reliably improve.

2) Repository structure (required)

Create a repo with this layout:

```
tootrasys/
  README.md
  requirements.txt
  .env.example

  specs/                    # the PDFs + MDs generated for modules 1-7
  canon/
    raw_pdfs/              # the source books/handbooks
    normalized/            # machine-usable canon JSON
    gold/                  # small "undeniable" gold examples (JSONL)
  datasets/
    jsonl/                 # training packs
    schemas/               # schema docs
    registry/              # datasets.yaml, mix configs
  src/
    tks/                   # runtime modules
    cli/                   # CLI entrypoints
  configs/                 # policy thresholds, train/eval configs
  replay_sets/             # frozen sets of episodes for regression
  logs/                    # event logs + traces (usually gitignored)
  scripts/                 # build canon, train, eval, replay, utilities
```

3) File placement checklist (what goes beside the PDFs)

3.1 Put the module docs here

- specs/module_1_executor_router/
- specs/module_2_verifier/
- specs/module_3_capability/
- specs/module_4_scoring/
- specs/module_5_eval_replay/
- specs/module_6_observability/
- specs/module_7_governance/

3.2 Put the core canon source PDFs here

- canon/raw_pdfs/
- Majik and the True Kabbalah
- Book of Kabbalistic Calculus

- Tootra math handbook
- Training curriculum
- Any other TKS/Tootra references

3.3 Put machine canon here (must be versioned + hashed)

- canon/normalized/
- canon_noetics_10.json
- canon_elements_40.json
- canon_acquisitions_22.json
- canon_foundations_7.json
- canon_subfoundations_28.json
- canon_operators.json
- canon_worlds_4.json
- canon_index.json (paths + versions + hashes)

3.4 Put training packs + schemas here

- datasets/jsonl/ (all JSONL packs)
- datasets/schemas/ (all schema .md files)

3.5 Put policy configs here

- configs/policy_baseline.yaml
- configs/policy_candidate.yaml
- configs/train.yaml
- configs/eval.yaml

4) CLI entrypoints (what commands exist)

Your CLI should expose these commands (names can vary, but keep the responsibilities identical):

4.1 Canon commands

- Build canon:
- `python scripts/canon/build_canon.py --in canon/raw_pdfs --out canon/normalized`

- Validate canon:
- `python scripts/canon/validate_canon.py --canon canon/normalized`
- Hash check:
- `python scripts/canon/hash_check.py --canon canon/normalized`

4.2 Runtime commands

- Run a single episode:
- `python -m tks.cli.run_episode --goal "... " --stack "1a:5b:2b" --mode Normal`
- Trace an episode:
- `python -m tks.cli.trace --episode ep_0001`

4.3 Evaluation / Replay commands

- Run test suites:
- `python -m tks.cli.eval --suite high_stakes`
- Run replay:
- `python -m tks.cli.replay --set replay_sets/rs_core.json --policy configs/policy_baseline.yaml`
- Compare policies:
- `python -m tks.cli.compare --set replay_sets/rs_core.json --baseline configs/policy_baseline.yaml --candidate configs/policy_candidate.yaml`

4.4 Training commands

- Mix datasets:
- `python scripts/train/mix_datasets.py --config datasets/registry/dataset_mix.yaml`
- Train:
- `python scripts/train/run_train.py --config configs/train.yaml`
- Evaluate model:
- `python scripts/eval/run_all.py --model checkpoints/latest`

5) Operational modes (Normal / High-Stakes / Critical)

High-stakes is a **behavioral gate**. It is not a suggestion.

Compute:

- $HS = (UK)(A^2)$
- U = uncertainty (0..1)
- K = stakes (0..1)
- A = alignment score (0..1)

Thresholds:

- $HS \geq 0.45 \rightarrow$ High-Stakes (PAUSE)
- $HS \geq 0.70 \rightarrow$ Critical (BLOCK)

Canonical expressions:

- High-Stakes: $(1:2:5^*) : (ASK: SLOW: VERIFY) + GATE(PAUSE)$
- Critical: $(1:2:5^*) : (ASK: SLOW: VERIFY) - EXECUTE$

Hard rule: In Critical, do not execute risky tools until verified + cleared.

6) Agent roles (how to split work across CLI agents)

Agent: agent - data

- Build and validate canon JSON
- Maintain gold examples
- Prevent definition drift

Agent: agent - be

- Implement runtime modules (Router/Executor/Verifier/Memory/Logs)
- Provide CLI commands

Agent: agent - arch

- Enforce module contracts and interfaces
- Keep policies consistent (gates, thresholds, caps)

Agent: agent - ml

- Manage dataset mixing, training, evaluation
- Maintain training pack registry

Agent: agent -eval

- Build test suites + regression sets
- Run replay comparisons before releases

Agent: agent -devops

- Packaging, environments, runbooks, CI wiring
- Ensures reproducibility

7) Implementation plan (phased)

Phase 1 — Thin vertical slice (1–2 days of work, conceptually)

Deliver:

- Router + Executor stubs
- Verifier parser + evidence binder minimal
- Logs (event stream)
- Replay runner that replays a stored episode with stubbed tool results

Acceptance:

- you can run `run_episode` and produce an event trace + memory packet + replay.

Phase 2 — Truth plumbing hardening

Deliver:

- operator substitution test
- consistency checker against canon JSON
- strict format validators (high-stakes packs)

Acceptance:

- “High-Stakes compliance suite” passes with 0 violations.

Phase 3 — Real tools (gradual enablement)

Start read-only:

- memory retrieve
- summarize evidence

Then:

- allowlisted search

Then:

- sandboxed code

Then:

- file ops with clearance tokens

Acceptance:

- each tool has a ToolProfile and failure modes logged.

Phase 4 — Learning + improvement

Deliver:

- outcome scoring
- rumination loop uses hard episodes
- reward-based template preference
- promotion gates use replay pass/fail

Acceptance:

- candidate policy shows improvement on replay sets without increasing violations.

8) Acceptance tests (must exist before “strong AI” claims)

8.1 Operator suite

- minimal pairs + substitution tests must pass

8.2 Nesting suite (Option A)

- $A:B:C == A:(B:C)$ always, normalization stable

8.3 Trigger routing suite

- FM-Female vs FM-Male vs ACBE vs MVR correct selection rate high
- hard negatives included

8.4 High-stakes suite

- Critical includes - EXECUTE and blocks execution
- Normal cases obey strict “(NORMAL)” when asked

8.5 Replay regression suite

- baseline vs candidate comparisons
- violations must not increase

9) Logging and debugging playbook

If something “feels off”, do this:

1) trace the episode and read:

- Mode, HS, Gate decisions
- RPM plan
- Router decisions
- Verifier PASS/FAIL and evidence links

2) Identify where drift occurred:

- parsing? evidence missing? canon contradiction? capability threshold too low?

3) Fix by policy, not by vibes:

- tighten thresholds
- add an eval case
- add a hard negative
- require evidence in that lane

10) Release process (safe iteration)

To change any policy/template:

1) create a candidate config (new version id)

2) run eval suites

3) run replay set comparisons

4) only deploy if:

- violations not increased

- reward and/or cost improved
- high-stakes compliance unchanged (or improved)

5) log deployment event + rollback plan

6) keep rollback one command away

11) Quickstart checklist (fastest way to start)

1) Put all PDFs + packs into the folder structure

2) Build canon JSON (even if manually at first)

3) Implement:

- Router decision function
- Verifier parser + gate
- Event log writer

4) Run a single episode and produce:

- trace + evidence id + TKS packet

5) Add one replay set and run baseline vs candidate

End of Runbook.